

ResuMatch AI — Presentation Recording Script

CAP 4630-002 | Total time target: 15-18 minutes

HOW TO USE THIS SCRIPT:

- Read it out loud naturally. Don't rush.
 - [PAUSE] = stop for 1-2 seconds before moving on
 - [CLICK] = advance to next slide or interact with the app
 - Practice once before recording. You'll sound way more confident.
-

SLIDE 1 — TITLE (~30 seconds)

"Hello everyone. Our project is called ResuMatch AI —
a resume and job description match scorer built using
Natural Language Processing and Machine Learning.

This is our final project for CAP 4630, Intro to Artificial Intelligence.
My name is [NAME], and I'm joined by [TEAMMATE 2] and [TEAMMATE 3].

Let's get into it."

[CLICK → Slide 2]

SLIDE 2 — OBJECTIVES (~1.5 minutes)

"So — what problem are we solving?

[point to Problem card]

If you've ever applied for a job, you've probably wondered:
'Does my resume actually match what they're looking for?'
Most people have no idea. They send out the same resume
to 50 different jobs and hope for the best.

[point to Why It Matters card]

This matters because over 75% of resumes are rejected
by automated Applicant Tracking Systems before a human
ever even reads them. So if your resume doesn't contain
the right keywords, you're eliminated before you even start.

[point to Our Goal card]

Our goal was to build a tool that solves this.
You paste in your resume and a job description,
and ResuMatch AI instantly gives you a match score
from 0 to 100%, shows you which keywords you already have,
and tells you exactly what's missing.

[PAUSE]

[CLICK → Slide 3]

SLIDE 3 — SYSTEM DESCRIPTION (~2 minutes)

"Let's talk about our dataset and our models.

[point to Dataset table]

We used a real dataset from Kaggle, published by Saugata Roy Arghya in 2023.
It contains 9,544 resumes across 28 different job categories —
everything from AI Engineer to Civil Engineer to HR Officer.

The dataset was unique because it already came with a `matched_score` column —
a numeric score from 0 to 1 representing how well each resume matched
its corresponding job description. We used that directly:
scores of 0.6 or above became our Match label,
and anything below 0.6 became No Match.
That gave us 6,971 matches and 2,573 non-matches.

[point to Models section]

For the actual AI, we built and compared three models.

First, TF-IDF Cosine Similarity as our baseline.
This converts text into vectors and measures the angle between them.
Simple, fast, and very interpretable.

Second, Random Forest Classifier — our primary model.
This creates a hundred decision trees and combines them.
It captured complex patterns between resumes and job descriptions
that simple cosine similarity couldn't.

And third, Logistic Regression as a comparison model.
All three gave us different perspectives on the same problem.

[CLICK → Slide 4]

SLIDE 4 — METHODOLOGY / PIPELINE (~2 minutes)

"Here's how the full pipeline works, step by step.

[point to Step 1]

We start with our raw data — 9,544 resumes across 28 categories.

[point to Step 2]

Then we clean the text. This means converting everything to lowercase,
removing brackets, URLs, and special characters, and filtering out
common stop words like 'the', 'and', 'is'. We did this entirely
with built-in Python — no external libraries needed.

[point to Step 3]

Next, we transform the cleaned text into TF-IDF vectors.
TF-IDF stands for Term Frequency-Inverse Document Frequency.
It figures out which words are important in a resume
relative to all the other resumes in the dataset.
We extracted 8,000 features using both single words and bigrams.

[point to Step 4]

Then we train our models. We used an 80/20 train-test split —
7,635 samples for training, 1,909 for testing.
The feature we fed the models was actually the element-wise
difference and product of the resume and job description vectors,
which captures how similar or different the two texts are.

[point to Steps 5 & 6]

Finally we evaluate using accuracy, precision, recall, and F1 score, and the results feed directly into our live Streamlit demo.

[point to bottom bar]

The full pre-processing pipeline is shown here at the bottom.

[CLICK → Slide 5]

SLIDE 5 — RESULTS: MODEL COMPARISON (~2 minutes)

"Let's look at the results.

[point to chart]

This bar chart compares all three models across four metrics: accuracy, precision, recall, and F1 score.

The orange bars are our TF-IDF cosine baseline.

It achieved 71% accuracy — decent for a simple unsupervised approach — but notice its recall is actually really high at 93%.

That means it's good at catching matches, but it also flags too many false positives, which hurts precision.

[point to teal bars]

The green bars are Random Forest. 84% accuracy, 89% F1 score. A significant improvement over the baseline.

[point to blue bars]

Logistic Regression performed similarly — also 84% accuracy, 89.6% F1. Both supervised models clearly outperformed the baseline.

[point to stat cards]

And to highlight the key numbers — 89.2% F1 for Random Forest, 89.6% for Logistic Regression, both trained on 9,544 real resume samples.

[PAUSE]

These are solid, realistic results for an NLP classification problem on real-world text data.

[CLICK → Slide 6]

SLIDE 6 — CONFUSION MATRIX + DISTRIBUTION (~1.5 minutes)

"Let's dig deeper into the Random Forest results.

[point to confusion matrix]

This is the confusion matrix on our test set of 1,909 samples.

321 true negatives — correctly identified as no match.

1,279 true positives — correctly identified as a match.

194 false positives — the model thought it was a match when it wasn't.

And 115 false negatives — missed matches.

[point to green insight box]

The key insight here is the recall — 91.8%.

For a job matching tool, recall matters more than precision.

You'd rather show someone a near-match than miss a real opportunity.

Our model gets that right.

[point to right chart]

Over on the right, this chart shows the original matched_score

distribution from the dataset itself. You can see the scores

clearly split into two groups on either side of our 0.6 threshold —

which tells us the threshold we chose was a natural boundary

in the data, not an arbitrary guess.

[CLICK → Slide 7]

SLIDE 7 — LIVE DEMO PREVIEW (~30 seconds)

"Here's a preview of the live demo.

This is our Streamlit web app — we'll show it live in just a moment.

You paste a resume on the left, a job description on the right,

click Analyze Match, and instantly get a score, matched keywords in green,

and missing keywords in red.

It works on any resume and any job description — it's not

limited to the training data at all.

Let's jump into the actual live demo now.

[CLICK → open Streamlit app in browser]

LIVE DEMO (~3-4 minutes)

[Open browser to localhost:8501]

"So here's ResuMatch AI running live.

[point to the sidebar]

On the left sidebar you can see information about the project and some pre-loaded example inputs.

[click 'Load HR Example' in sidebar]

I'll click the HR example to load a sample resume and job description.

[point to left text box]

On the left we have Maria Garcia — an HR Specialist with experience in talent acquisition, onboarding, payroll, HRIS systems like Workday, and performance management.

[point to right text box]

On the right we have a job posting for an HR Manager at a healthcare organization — looking for someone with HRIS, payroll, compliance, and benefits administration experience.

[click Analyze Match button]

Now I'll click Analyze Match.

[PAUSE — let it load]

[point to score]

And there it is — a match score. You can see the percentage at the top, along with whether it's a Strong, Partial, or Weak match.

[point to green keywords]

Down here in green are the keywords your resume already has that the job description is looking for — things like talent acquisition, onboarding, payroll, Workday.

[point to red keywords]

In red are the keywords that are missing.

These are the specific words you'd want to add to your resume before applying to this job.

[point to recommendation]

And at the bottom there's an automatic recommendation telling you exactly what to do to improve your score.

[PAUSE]

Let me try one more — the Data Science example.

[click 'Load Data Science Example' in sidebar]

[click Analyze Match]

[point to higher score]

This one scores higher because the resume and job description share a lot of specific technical keywords — Python, machine learning, TensorFlow, scikit-learn.

That's the power of TF-IDF — the more domain-specific language your resume shares with the job posting, the higher your match score.

[CLICK → back to slides → Slide 8]

SLIDE 8 — OBSERVATIONS & CHALLENGES (~1.5 minutes)

"Now let's talk about what worked and what was challenging.

[point to green side]

What worked really well:

The TF-IDF score distributions were clearly separable — matched pairs scored noticeably higher than non-matched pairs, which is exactly what you want.

Both the Random Forest and Logistic Regression significantly outperformed the cosine baseline — jumping from 71% to 84% accuracy.

The dataset's existing `matched_score` column was incredibly useful — it gave us a real-world label without any manual annotation.

And the Streamlit app is genuinely usable. It works on any text you give it, in real time, with zero lag.

[point to orange side]

The challenges:

Our biggest technical issue was Python 3.14 compatibility.

The pinned library versions in our requirements file couldn't build on Python 3.14. The fix was simple — remove the version pins and let pip figure it out automatically.

We also hit a network restriction with NLTK, which couldn't download its data files. We replaced it entirely with a built-in Python stopwords list and regex-based cleaning — which actually made the code simpler and more portable.

Finally, the dataset has a class imbalance —

73% matches versus 27% no-matches.

We handled this by using a stratified train-test split to ensure both classes were fairly represented in evaluation.

[CLICK → Slide 9]

SLIDE 9 — CONCLUSION (~1.5 minutes)

"Let's wrap up.

[point to takeaways]

Our four key takeaways:

One — TF-IDF is a fast, interpretable way to measure text similarity between resumes and job descriptions.

It's not perfect, but it's a strong starting point.

Two — Both Random Forest and Logistic Regression reached 84% accuracy on nearly 10,000 real resumes.

That's meaningful performance on messy, real-world text data.

Three — The dataset's `matched_score` column was a major advantage. It provided real signal without any manual labeling work.

Four — Wrapping the model in a Streamlit app turns research into something anyone can actually use.

That gap between model and product matters.

[point to future work]

For future work, we'd love to integrate SBERT — Sentence BERT — which understands semantic meaning, not just keyword overlap. So it would know that 'ML' and 'machine learning' mean the same thing.

We'd also want to add a recruiter mode where you can upload multiple resumes and rank them against one job posting, and eventually deploy this publicly on Streamlit Cloud.

[PAUSE]

Thank you for watching. Our full code, dataset pipeline, and this presentation are all available on our GitHub repository.

The link is in the description."

[END RECORDING]

TIMING GUIDE

Slide 1 — Title	~0:30
Slide 2 — Objectives	~1:30
Slide 3 — System Desc.	~2:00
Slide 4 — Pipeline	~2:00
Slide 5 — Results	~2:00
Slide 6 — Analysis	~1:30
Slide 7 — Demo intro	~0:30
Live Demo	~3:30
Slide 8 — Observations	~1:30
Slide 9 — Conclusion	~1:30

TOTAL	~16:30
-------	--------

RECORDING TIPS

1. Open your slides in full screen FIRST before starting OBS

2. Have the Streamlit app already running at localhost:8501 before you start recording (run it in a terminal, then minimize)
3. Speak slower than feels natural — recordings always feel faster
4. If you mess up a sentence, just pause and repeat it — you can trim mistakes in any video editor
5. Make sure your mic is working before you hit record
6. Windows + G opens Xbox Game Bar if you don't have OBS — it can record your screen for free with no setup